

JavaScript Practice Questions Answers

Basic:

1. What is JavaScript, and how is it different from Java?

- JavaScript is a scripting/programming language mainly used to make web pages interactive.
 - Java is a different programming language used for applications, backend systems, Android, etc.
-

2. What is the difference between var, let, and const?

- var: Function-scoped, can be redeclared and reassigned.
 - let: Block-scoped, can be reassigned but not redeclared in the same scope.
 - const: Block-scoped, cannot be reassigned or redeclared.
-

3. What are primitive and non-primitive data types?

- Primitive: Stores a single value. Example: string, number, boolean, null, undefined, bigint, symbol.
 - Non-primitive: Stores collections/references. Example: objects, arrays, functions.
-

4. What is the difference between null and undefined?

- null: Intentionally represents no value.
 - undefined: A variable has been declared but has no assigned value.
-

5. What is the difference between == and ===?

- ==: Compares values after type conversion.
 - ===: Compares both value and data type.
-

6. What are truthy and falsy values?

- Truthy: Values treated as true in a condition.

- Falsy: Values treated as false.
 - Falsy examples: false, 0, "", null, undefined, NaN.
-

7. What is type coercion?

- Automatic or explicit conversion of one data type into another.
 - Example: "5" + 2 gives "52".
-

8. What is NaN, and how can you check for it?

- NaN means "Not a Number".
 - Use Number.isNaN(value) to check for it.
-

9. What is the difference between a function declaration and a function expression?

- Function declaration: Defined using the function keyword and can be called before its definition.
 - Function expression: A function stored in a variable.
-

10. What is an arrow function?

- A shorter way to write functions using =>.
 - Example: const add = (a, b) => a + b;
-

11. What are template literals?

- Strings written using backticks (`) that allow variables and expressions inside \${}.
 - Example: Hello \${name}
-

12. What are default parameters?

- Values automatically used when an argument is not provided.
 - Example: function greet(name = "Guest") {}
-

13. What are rest and spread operators?

- Rest (...): Collects multiple values into an array.

- Spread (...): Expands an array/object into individual values.
-

14. What is destructuring?

- Extracting values from arrays or objects into variables.
 - Example: `const [a, b] = [10, 20];`
-

15. What is strict mode?

- Strict mode makes JavaScript code safer by enforcing stricter rules.
 - Enabled using: `"use strict";`
-

Scope and Functions

16. What is scope in JavaScript?

- Scope determines where a variable can be accessed in a program.
-

17. What is the difference between global, function, and block scope?

- Global scope: Accessible throughout the program.
 - Function scope: Accessible only inside a function.
 - Block scope: Accessible only inside { }, mainly with let and const.
-

18. What is lexical scope?

- Lexical scope means a function can access variables from the place where it was defined.
-

19. What is a closure? Give a practical example.

- A closure is a function that remembers variables from its outer function even after the outer function has finished.

Example:

```
function counter() {  
  let count = 0;  
  return () => ++count;  
}  
  
const c = counter();  
console.log(c()); // Output: 1  
console.log(c()); // Output: 2
```

20. What is hoisting?

- Hoisting is JavaScript's behavior of processing declarations before executing the code.
-

21. How are var, let, and const hoisted differently?

- var: Hoisted and initialized with undefined.
- let: Hoisted but not initialized.

- `const`: Hoisted but not initialized.
-

22. What is the Temporal Dead Zone?

- The Temporal Dead Zone (TDZ) is the period between entering a scope and declaring a `let` or `const` variable.
 - Accessing the variable during this period causes an error.
-

23. What is a callback function?

- A callback is a function passed as an argument to another function.

Example:

```
function greet(name, callback) {  
  callback(name);  
}  
  
greet("John", console.log);  
// Output: John
```

24. What is a higher-order function?

- A function that takes another function as an argument or returns a function.
 - Example: `map()`, `filter()`, and `reduce()`.
-

25. What is a pure function?

- A pure function always gives the same output for the same input and does not change outside data.

Example:

```
function add(a, b) {  
  return a + b;  
}
```

26. What is an IIFE?

- IIFE stands for Immediately Invoked Function Expression.
- It runs immediately after it is created.

Example:

```
(function () {  
  console.log("Hello");  
})();
```

// Output: Hello

27. What is function currying?

- Currying converts a function with multiple arguments into a sequence of functions with one argument each.

Example:

```
function add(a) {  
  return function (b) {  
    return a + b;  
  };  
}
```

```
console.log(add(5)(3));
```

// Output: 8

28. What is recursion?

- Recursion is when a function calls itself until a stopping condition is reached.

Example:

```
function count(n) {  
  if (n === 0) return;  
  console.log(n);  
  count(n - 1);  
}
```

```
count(3);
```

// Output:

// 3

// 2

// 1

29. What does this represent?

- this refers to the object associated with the current function call.
- Its value depends on how the function is called.

30. How does this behave inside an arrow function?

- Arrow functions do not have their own this.
- They use this from their surrounding (lexical) scope.

31. Explain call(), apply(), and bind().

- call(): Calls a function with a specified this value and arguments separately.
- apply(): Calls a function with a specified this value and arguments as an array.
- bind(): Creates a new function with a specified this value.

Example:

```
function greet(city) {  
  console.log(this.name + " from " + city);  
}
```

```
const person = { name: "John" };  
  
greet.call(person, "Delhi");  
// Output: John from Delhi
```

```
greet.apply(person, ["Delhi"]);  
// Output: John from Delhi
```

```
const newGreet = greet.bind(person);  
newGreet("Delhi");  
// Output: John from Delhi
```

Arrays and Objects

32. What is the difference between `map()`, `filter()`, and `reduce()`?

- `map()`: Creates a new array by transforming each element.
 - `filter()`: Creates a new array with elements that pass a condition.
 - `reduce()`: Reduces an array to a single value.
-

33. What is the difference between `forEach()` and `map()`?

- `forEach()`: Executes a function for each element but does not return a new array.
 - `map()`: Executes a function for each element and returns a new array.
-

34. What is the difference between `find()` and `filter()`?

- `find()`: Returns the first element that matches the condition.
 - `filter()`: Returns all elements that match the condition.
-

35. What is the difference between `slice()` and `splice()`?

- `slice()`: Extracts elements without changing the original array.
 - `splice()`: Adds, removes, or replaces elements and changes the original array.
-

36. Which array methods mutate the original array?

Common mutating methods are:

- `push()`
- `pop()`
- `shift()`
- `unshift()`
- `splice()`
- `reverse()`
- `sort()`

- fill()
 - copyWithin()
-

37. What is object destructuring?

- Object destructuring extracts properties from an object into variables.

Example:

```
const person = { name: "John", age: 25 };
```

```
const { name, age } = person;
```

```
console.log(name);
```

```
// Output: John
```

38. What is the difference between shallow copy and deep copy?

- Shallow copy: Copies the object, but nested objects are still shared.
 - Deep copy: Copies the object and its nested objects independently.
-

39. How can you clone an object?

Common ways:

- Spread operator: { ...obj }
- Object.assign({}, obj)
- structuredClone(obj) for a deep copy.

Example:

```
const copy = { ...person };
```

40. What are object references?

- Objects are stored and accessed through references.
- Two variables can refer to the same object.

Example:

```
let a = { name: "John" };
```

```
let b = a;
```

```
b.name = "Alex";
```

```
console.log(a.name);
```

```
// Output: Alex
```

41. What is immutability, and why is it useful in React?

- Immutability means not directly changing existing data.
 - In React, it helps detect state changes and makes updates more predictable.
-

42. What is optional chaining?

- Optional chaining (?.) safely accesses properties without causing an error if a value is null or undefined.

Example:

```
console.log(user?.address?.city);
```

43. What is nullish coalescing?

- The nullish coalescing operator (??) provides a default value when the left side is null or undefined.

Example:

```
let name = null;
```

```
console.log(name ?? "Guest");
```

```
// Output: Guest
```

44. What is the difference between `Object.keys()`, `Object.values()`, and `Object.entries()`?

- `Object.keys()`: Returns an array of object property names.
- `Object.values()`: Returns an array of property values.
- `Object.entries()`: Returns an array of [key, value] pairs.

Example:

```
const person = { name: "John", age: 25 };
```

```
console.log(Object.keys(person));
```

```
// Output: ["name", "age"]
```

```
console.log(Object.values(person));
```

```
// Output: ["John", 25]
```

```
console.log(Object.entries(person));  
// Output: [["name", "John"], ["age", 25]]
```

Asynchrone JavaScript

45. What is synchronous JavaScript?

- Synchronous JavaScript executes code one task at a time, in order.
 - Each task waits for the previous task to finish.
-

46. What is asynchronous JavaScript?

- Asynchronous JavaScript allows tasks to run without blocking the execution of other code.
 - Example: `setTimeout()`, Promises, `fetch()`.
-

47. What is the call stack?

- The call stack keeps track of functions currently being executed.
 - JavaScript executes functions from the top of the stack.
-

48. What are Browser Web APIs?

- Web APIs are browser-provided features that JavaScript can use.
 - Examples: DOM, `setTimeout()`, `fetch()`, `localStorage`.
-

49. What is the event loop?

- The event loop checks whether the call stack is empty and moves waiting tasks into it for execution.
 - It helps JavaScript handle asynchronous operations.
-

50. What is the difference between microtasks and macrotasks?

- Microtasks: Promise callbacks, `queueMicrotask()`.
 - Macrotasks: `setTimeout()`, `setInterval()`, DOM events.
 - Microtasks are generally executed before the next macrotask.
-

51. What is callback hell?

- Callback hell occurs when many nested callbacks make code difficult to read and maintain.

Example:

```
doSomething() => {  
  doSomethingElse() => {  
    doAnotherThing() => {  
      // More code...  
    };  
  };  
};
```

52. What is a Promise?

- A Promise represents the eventual success or failure of an asynchronous operation.
 - It can be handled using `.then()`, `.catch()`, and `.finally()`.
-

53. What are the states of a Promise?

A Promise has three states:

- Pending: Operation is still running.
 - Fulfilled: Operation completed successfully.
 - Rejected: Operation failed.
-

54. What is Promise chaining?

- Promise chaining means using multiple `.then()` methods one after another.
- The result of one operation can be passed to the next.

Example:

```
getData()  
  .then(data => processData(data))  
  .then(result => console.log(result))  
  .catch(error => console.log(error));
```

55. What is the difference between `.then()` and `await`?

- `.then()`: Handles the result of a Promise using callback functions.
- `await`: Waits for a Promise to settle inside an `async` function and makes asynchronous code easier to read.

56. Explain Promise.all().

- Runs multiple Promises together.
- Resolves when ALL Promises succeed.
- Rejects immediately when any Promise rejects.

Example:

```
Promise.all([p1, p2, p3])  
.then(results => console.log(results));
```

57. Explain Promise.allSettled().

- Waits for ALL Promises to finish.
- Returns the result of both fulfilled and rejected Promises.

Example:

```
Promise.allSettled([p1, p2, p3])  
.then(results => console.log(results));
```

58. Explain Promise.race().

- Returns the result of the first Promise to settle.
- It can be either fulfilled or rejected.

Example:

```
Promise.race([p1, p2])  
.then(result => console.log(result));
```

59. Explain Promise.any().

- Returns the first Promise that fulfills.
- It ignores rejected Promises unless all Promises reject.

Example:

```
Promise.any([p1, p2, p3])  
.then(result => console.log(result));
```

60. How do you handle errors with async/await?

- Use try...catch to handle errors.

Example:

```
async function getData() {  
  try {  
    const data = await fetchData();  
    console.log(data);  
  } catch (error) {  
    console.log(error);  
  }  
}
```

61. What happens when one Promise fails inside Promise.all()?

- Promise.all() immediately rejects when any Promise rejects.
- It does not return the successful results from the other Promises.

Example:

```
Promise.all([p1, p2, p3])  
  .then(result => console.log(result))  
  .catch(error => console.log(error));
```

Advanced & Practical Concepts

62. What is prototype inheritance?

- Prototype inheritance allows an object to access properties and methods from another object through its prototype.
-

63. What is the prototype chain?

- The prototype chain is the chain of objects JavaScript searches when looking for a property or method.
 - It continues until it reaches null.
-

64. What is the difference between prototype and **proto**?

- **prototype**: A property of constructor functions used to define properties/methods for created objects.
 - **proto**: A property that refers to an object's prototype.
-

65. What are iterators?

- Iterators are objects that allow you to access elements one at a time.
- They use the `next()` method.

Example:

```
let arr = [10, 20];  
let iterator = arrSymbol.iterator;  
  
console.log(iterator.next());  
// Output: { value: 10, done: false }
```

66. What are generators?

- Generators are special functions that can pause and resume execution.
- They are created using `function*` and use `yield`.

Example:

```
function* numbers() {  
  yield 1;  
}
```

```
yield 2;
}  
  
let gen = numbers();  
console.log(gen.next().value);  
// Output: 1
```

67. What is debouncing?

- Debouncing delays a function until the user stops performing an action for a specified time.
 - Commonly used for search boxes and input events.
-

68. What is throttling?

- Throttling limits how often a function can execute within a specific time period.
 - Commonly used for scroll and resize events.
-

69. What is event bubbling?

- Event bubbling means an event starts from the target element and moves upward through its parent elements.

Example:

button → div → body → document

70. What is event capturing?

- Event capturing means an event travels from the outermost parent toward the target element.

Example:

document → body → div → button

71. What is event delegation?

- Event delegation means adding one event listener to a parent instead of adding separate listeners to each child.
 - It works using event bubbling.
-

72. What is garbage collection?

- Garbage collection automatically removes objects that are no longer reachable or needed.
 - It helps free unused memory.
-

73. What are JavaScript modules?

- Modules allow JavaScript code to be divided into separate files.
- They use export and import.

Example:

```
// math.js
export const add = (a, b) => a + b;

// app.js
import { add } from "./math.js";
```

74. What is the difference between named and default exports?

- Named export: Multiple values can be exported and imported using their names.
- Default export: One default value can be exported from a module.

Example:

```
// Named
export const name = "John";
import { name } from "./file.js";

// Default
export default name;
import name from "./file.js";
```

75. What is a dynamic import?

- Dynamic import loads a module when it is needed instead of loading it immediately.
- It returns a Promise.

Example:

```
import("./math.js")
.then(module => console.log(module));
```

76. What are Set and Map?

- Set: Stores unique values.
- Map: Stores key-value pairs.

Example:

```
let set = new Set([1, 2, 2]);
```

```
console.log(set);
```

```
// Output: Set(2) {1, 2}
```

```
let map = new Map();
```

```
map.set("name", "John");
```

77. How is a Map different from a normal object?

- Map can use any type as a key.
- Object keys are generally strings or symbols.
- Map also provides methods like set(), get(), has(), and delete().

78. What is memoization?

- Memoization stores the results of expensive function calls.
- If the same input is used again, the stored result can be returned.

79. What is local storage?

- localStorage stores data in the browser as key-value pairs.
- Data remains even after the browser is closed.

Example:

```
localStorage.setItem("name", "John");
```

```
console.log(localStorage.getItem("name"));
```

```
// Output: John
```

80. What is the difference between local storage, session storage, and cookies?

- localStorage: Stores data until it is manually removed.
 - sessionStorage: Stores data for the current browser tab/session.
 - Cookies: Small pieces of data that can be sent to the server with HTTP requests.
-

Output-based interview questions

Question 1: Hoisting

```
console.log(a);
```

```
var a = 10;
```

Output:

undefined

Why?

- var declarations are hoisted to the top.
- The variable is initialized with undefined.

Equivalent to:

```
var a;
```

```
console.log(a);
```

```
a = 10;
```

Question 2: Temporal Dead Zone

```
console.log(a);
```

```
let a = 10;
```

Output:

ReferenceError

Why?

- let is hoisted but not initialized.
 - Accessing it before its declaration is inside the **Temporal Dead Zone (TDZ)**.
-

Question 3: Type Coercion

```
console.log(1 + "2");
```

```
console.log("5" - 2);
```

```
console.log(true + 1);
```

Output:

12

3

2

Why?

- `1 + "2" → Number is converted to String → "12"`
 - `"5" - 2 → String is converted to Number → 3`
 - `true + 1 → true is converted to 1 → 2`
-

Question 4: Equality

```
console.log(0 == false);
```

```
console.log(0 === false);
```

```
console.log(null == undefined);
```

```
console.log(null === undefined);
```

Output:

true

false

true

false

Why?

- `==` allows type conversion.
 - `===` checks both value and type.
 - `null == undefined` is a special equality rule in JavaScript.
 - `null` and `undefined` have different types, so `===` returns false.
-

Question 5: Object Reference

```
const user1 = {  
  name: "Meghana"  
};
```

```
const user2 = user1;  
user2.name = "Deepak";  
console.log(user1.name);
```

Output:

Deepak

Why?

- user1 and user2 refer to the **same object**.
 - Changing the object through user2 also changes what user1 sees.
-

Question 6: Shallow Copy

```
const original = {  
  name: "Meghana",  
  address: {  
    city: "Bangalore"  
  }  
};
```

```
const copied = { ...original };
```

```
copied.address.city = "Mysore";
```

```
console.log(original.address.city);
```

Output:

Mysore

Why?

- Spread syntax creates a **shallow copy**.
- The nested address object is still shared between original and copied.
- Therefore, changing copied.address.city also changes original.address.city.

Question 7: Closure

```
function outer() {  
  let count = 0;  
  
  return function () {  
    count++;  
    console.log(count);  
  };  
}
```

```
const counter = outer();
```

```
counter();
```

```
counter();
```

```
counter();
```

Output:

```
1
```

```
2
```

```
3
```

Why?

- The inner function forms a **closure** over the count variable.
- It remembers count even after outer() has finished executing.
- Each call increases the same count value.

Question 8: setTimeout()

```
console.log("A");
```

```
setTimeout(() => {
```

```
console.log("B");  
}, 0);
```

```
console.log("C");
```

Output:

A

C

B

Why?

- Synchronous code runs first.
 - `setTimeout()` is handled asynchronously.
 - The callback runs after the current code finishes.
-

Question 9: Promise and Timer

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Timer");  
}, 0);
```

```
Promise.resolve().then(() => {  
  console.log("Promise");  
});
```

```
console.log("End");
```

Output:

Start

End

Promise

Timer

Why?

- Start and End are synchronous.
- Promise callbacks are **microtasks**.
- setTimeout() callbacks are **macrotasks**.
- Microtasks are executed before macrotasks.

Order:

Synchronous → Microtasks → Macrotasks

Question 10: var inside a loop

```
for (var i = 1; i <= 3; i++) {  
  setTimeout(() => {  
    console.log(i);  
  }, 1000);  
}
```

Output:

4
4
4

Why?

- var is function-scoped, so all callbacks share the same i.
- By the time the callbacks execute, the loop has finished and i is 4.

Fix using let:

```
for (let i = 1; i <= 3; i++) {  
  setTimeout(() => {  
    console.log(i);  
  }, 1000);  
}
```

Output:

1

2

3

Question 11: this

```
const user = {  
  name: "Meghana",  
  
  regularFunction() {  
    console.log(this.name);  
  },  
  
  arrowFunction: () => {  
    console.log(this.name);  
  }  
};
```

```
user.regularFunction();
```

```
user.arrowFunction();
```

Output:

Meghana

undefined

Why?

- Regular function: this refers to the user object when called as user.regularFunction().
- Arrow function: It does not have its own this; it uses this from its surrounding scope.

The second output can depend on the surrounding execution environment, but in the usual module/global context it is undefined.

Question 12: Array Methods

```
const numbers = [1, 2, 3, 4, 5];
```

```
const result = numbers
```

```
.filter(number => number % 2 === 0)
```

```
.map(number => number * 10);
```

```
console.log(result);
```

Output:

```
[20, 40]
```

Why?

- filter() selects even numbers → [2, 4]
 - map() multiplies each by 10 → [20, 40]
-

Question 13: map() with parseInt()

```
console.log(["1", "2", "3"].map(parseInt));
```

Output:

```
[1, NaN, NaN]
```

Why?

map() passes **value, index, and array** to parseInt().

So it becomes:

```
parseInt("1", 0); // 1
```

```
parseInt("2", 1); // NaN
```

```
parseInt("3", 2); // NaN
```

The second argument is treated as the **radix/base**.

Question 14: Promise Error Handling

```
Promise.resolve(10)
```

```
.then(value => value * 2)
```

```
.then(() => {
```

```
    throw new Error("Something went wrong");
  })
  .catch(() => 50)
  .then(value => console.log(value));
```

Output:

50

Why?

- $10 * 2 \rightarrow 20$
 - The next `.then()` throws an error.
 - `.catch()` handles the error and returns 50.
 - The final `.then()` receives 50.
-

Question 15: Generator

```
function* generateNumbers() {
  yield 1;
  yield 2;
  return 3;
}
```

```
const generator = generateNumbers();
```

```
console.log(generator.next());
console.log(generator.next());
console.log(generator.next());
console.log(generator.next());
```

Output:

```
{ value: 1, done: false }
{ value: 2, done: false }
{ value: 3, done: true }
```

```
{ value: undefined, done: true }
```

Why?

- First next() → reaches yield 1.
- Second next() → reaches yield 2.
- Third next() → reaches return 3 and the generator finishes.
- Fourth next() → generator is already finished.